UNIVERSIDAD PRIVADA DEL NORTE

Facultad de Ingeniería

Ingeniería de Sistemas Computacionales

PRÁCTICA DE CAMPO – SEMANA 5

Programación Dinámica aplicada al caso SmartOptimizer S.A.C.

Curso: Análisis de Algoritmos y Estrategias de Programación

Docente: Herber Edinson De La Cruz Pampañaupa

Grupo: N.° 7

Fecha de entrega: 15 de noviembre de 2026

Integrantes que participaron

Apellidos y nombres	Participación
BENITES VILLAVICENCIO, Paolo Sergio	100 %
CANO ROJAS, Mijael Deyvi	100 %
CASSINA MUÑOZ, Andres Guillermo	100 %
QUISPE HUAMANI, Nayda	100 %
ZUTTA LOPEZ, Deyvith Anderson	100 %

Lima – Perú, 2026

1. Introducción

SmartOptimizer S.A.C. desarrolla soluciones tecnológicas para empresas cuyos clientes toman decisiones con procedimientos manuales que obligan a evaluar muchas alternativas. La empresa pidió un sistema en Python, con interfaz gráfica en Tkinter, que resuelva tres problemas de optimización usando Programación Dinámica (PD): el costo mínimo de una ruta de desplazamiento, el número de formas en que un técnico puede alcanzar un nivel de certificación y la selección de proyectos con mayor beneficio cuando no se pueden elegir dos consecutivos.

La Programación Dinámica resuelve un problema dividiéndolo en subproblemas que se repiten, guarda la solución de cada uno en una tabla y la reutiliza en lugar de volver a calcularla. Se aplica cuando el problema tiene subestructura óptima (la solución óptima se arma con soluciones óptimas de subproblemas) y subproblemas superpuestos (los mismos cálculos aparecen muchas veces). En los tres casos se usó el enfoque ascendente o de tabulación (bottom-up): se empieza por el caso base y se llena la tabla hasta llegar a la respuesta (Cormen et al., 2022).

1.1 Objetivo

Implementar en Python una aplicación Tkinter con tres módulos seleccionables que aplique Programación Dinámica para resolver las casuísticas de SmartOptimizer S.A.C., mostrando la tabla de resultados, la relación de recurrencia y el análisis de cada solución.

1.2 Estructura de la solución

El código está separado en dos archivos para que la lógica pueda probarse sin abrir la ventana:

- src/dp_logica.py: las tres funciones de Programación Dinámica, comentadas.
- src/smartoptimizer_app.py: la interfaz Tkinter con una pestaña por módulo. Cada pestaña permite cambiar los datos, recalcular y ver la tabla DP, el gráfico de la solución y la explicación.

Para ejecutarla: python src/smartoptimizer_app.py (requiere Python 3.10 o superior; Tkinter viene incluido).

2. Casuística 01: Optimización de rutas de desplazamiento

2.1 Planteamiento

Un técnico debe ir de la posición 0 a la 10 y en cada movimiento puede avanzar 1, 2 o 3 posiciones, con costos de S/ 10, S/ 18 y S/ 25. Se busca el costo mínimo acumulado. Si dp[i] es el costo mínimo para llegar a la posición i, a i solo se puede llegar desde i − 1, i − 2 o i − 3, así que basta con quedarse con la opción más barata:

$$dp[i] = \min(\, dp[i-1] + 10,\ dp[i-2] + 18,\ dp[i-3] + 25\,), \quad \text{con } dp[0] = 0$$

Además de dp[i] se guarda qué paso dio el mínimo, para reconstruir la ruta recorriendo la tabla desde el destino hacia atrás.

2.2 Implementación

```
def ruta_costo_minimo(inicio, destino, costos):
    """Costo mínimo para ir de `inicio` a `destino`.

    costos: dict {pasos: costo}, p. ej. {1: 10, 2: 18, 3: 25}
    Recurrencia: dp[i] = min(dp[i - p] + costo[p])  para cada paso p con i - p >= inicio
    Caso base:   dp[inicio] = 0
    Complejidad: O(n * k) en tiempo y O(n) en espacio (n = posiciones, k = tipos de paso).
    Devuelve (costo_minimo, tabla_dp, decision, camino).
    """
    if destino < inicio:
        raise ValueError("El destino debe ser mayor o igual que la posición inicial.")
    n = destino - inicio
    INF = float("inf")
    dp = [INF] * (n + 1)          # dp[j] = costo mínimo para llegar a la posición inicio + j
    decision = [None] * (n + 1)   # paso usado para llegar a j (para reconstruir el camino)
    dp[0] = 0
    for j in range(1, n + 1):
        for pasos, costo in sorted(costos.items()):
            if j - pasos >= 0 and dp[j - pasos] + costo < dp[j]:
                dp[j] = dp[j - pasos] + costo
                decision[j] = pasos

    # Reconstrucción del camino desde el destino hacia atrás
    camino, j = [], n
    while j > 0:
        p = decision[j]
        camino.append((inicio + j - p, inicio + j, p, costos[p]))
        j -= p
    camino.reverse()
    return dp[n], dp, decision, camino
```

2.3 Resultados

Tabla 1

Tabla DP del costo mínimo por posición

Posición i	dp[i] (S/)	Último paso	Viene de
0	0	—	inicio
1	10	+1	0
2	18	+2	0
3	25	+3	0
4	35	+1	3
5	43	+2	3
6	50	+3	3
7	60	+1	6
8	68	+2	6
9	75	+3	6
10	85	+1	9

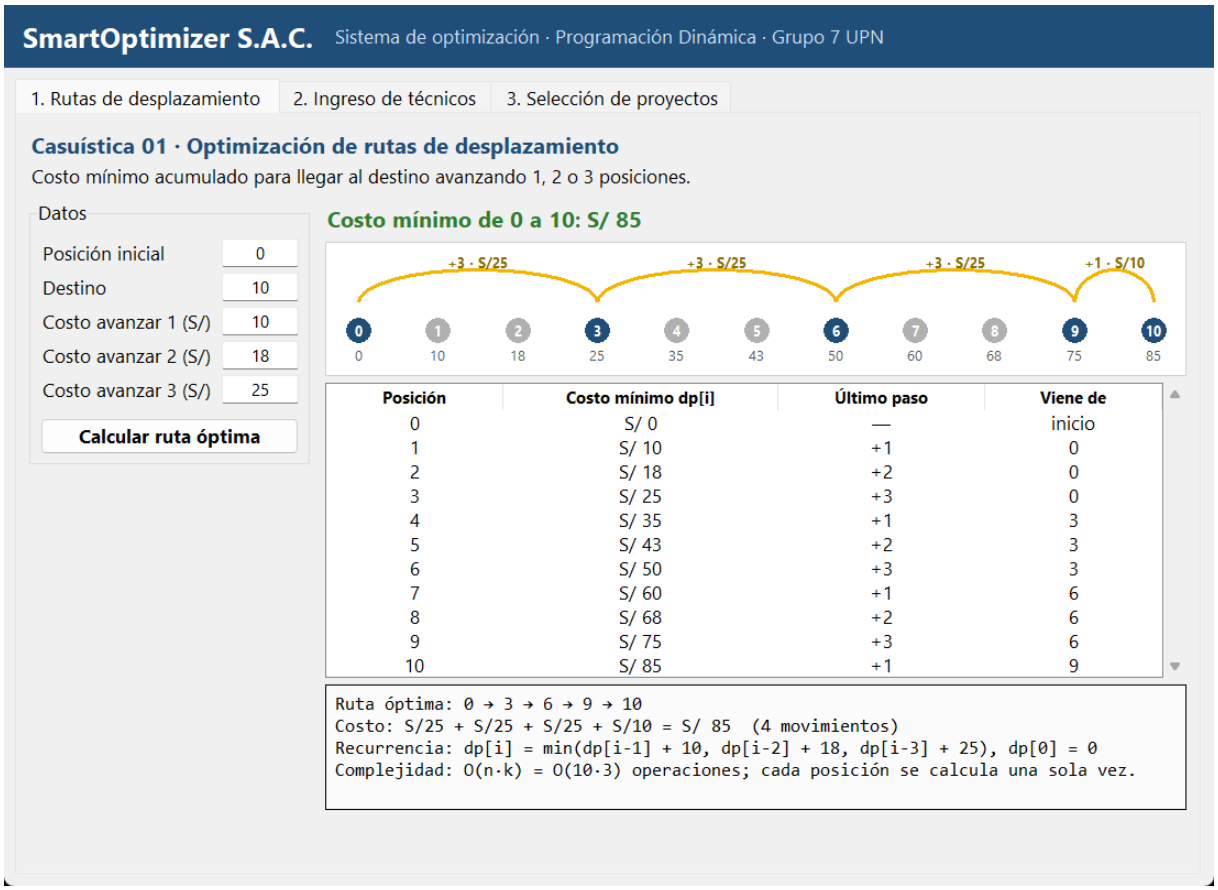
Nota. Elaboración propia.

El costo mínimo para llegar de la posición 0 a la 10 es S/ 85, con la ruta 0 → 3 → 6 → 9 → 10: tres saltos de 3 posiciones (S/ 25 cada uno) y un último avance de 1 posición (S/ 10). El salto de 3 es el más conveniente porque cuesta S/ 8,33 por posición, frente a S/ 9 del salto de 2 y S/ 10 del de 1; la PD lo confirma sin probar todas las combinaciones. Hay empates (por ejemplo, a la

posición 4 se llega con S/ 35 por 0→1→4 o por 0→3→4), y la tabla conserva solo uno, porque cualquiera de ellos da el mismo costo mínimo.

Figura 1

Módulo 1 de la aplicación: ruta óptima, tabla DP y recurrencia



Nota. Elaboración propia.

2.4 Complejidad

Cada una de las n posiciones se calcula una vez revisando k = 3 tipos de paso, por lo que el tiempo es O(n·k) y el espacio O(n). Para n = 10 son 30 comparaciones. Una búsqueda exhaustiva tendría que recorrer todas las secuencias de pasos que suman 10, que son 274 y crecen de forma exponencial con n.

3. Casuística 02: Optimización del ingreso de técnicos

3.1 Planteamiento

Un técnico debe superar 8 niveles de certificación y puede avanzar 1, 2 o 3 niveles a la vez. Se pide cuántas formas distintas existen de llegar al nivel 8. Toda forma de llegar al nivel n termina con un salto de 1, 2 o 3, de modo que el total es la suma de las formas de llegar a los tres niveles anteriores:

$$f(n) = f(n-1) + f(n-2) + f(n-3), \quad \text{con } f(0) = 1 \text{ y } f(n) = 0 \text{ si } n < 0$$

El caso base f(0) = 1 representa la única forma de estar en el nivel inicial: no haber dado ningún salto. Esta recurrencia corresponde a la sucesión de Tribonacci.

3.2 Implementación

```
def formas_de_llegar(niveles, saltos=(1, 2, 3)):
    """Número de formas distintas de llegar al nivel `niveles` avanzando con `saltos`.

    Recurrencia: f(n) = f(n-1) + f(n-2) + f(n-3)   (con saltos 1, 2 y 3)
    Caso base:   f(0) = 1 (una forma de "no moverse"); f(n) = 0 si n < 0
    Complejidad: O(n * k) en tiempo y O(n) en espacio.
    Devuelve (total, tabla_f, detalle) donde detalle[i] es el texto del cálculo de f(i).
    """
    if niveles < 0:
        raise ValueError("La cantidad de niveles no puede ser negativa.")
    f = [0] * (niveles + 1)
    f[0] = 1
    detalle = ["f(0) = 1  (caso base)"]
    for i in range(1, niveles + 1):
        partes, nombres = [], []
        for s in saltos:
            if i - s >= 0:
                f[i] += f[i - s]
                partes.append(str(f[i - s]))
                nombres.append(f"f({i - s})")
        detalle.append(f"f({i}) = {' + '.join(nombres)} = {' + '.join(partes)} = {f[i]}")
    return f[niveles], f, detalle
```

3.3 Resultados parciales

Tabla 2

Número de formas de llegar a cada nivel

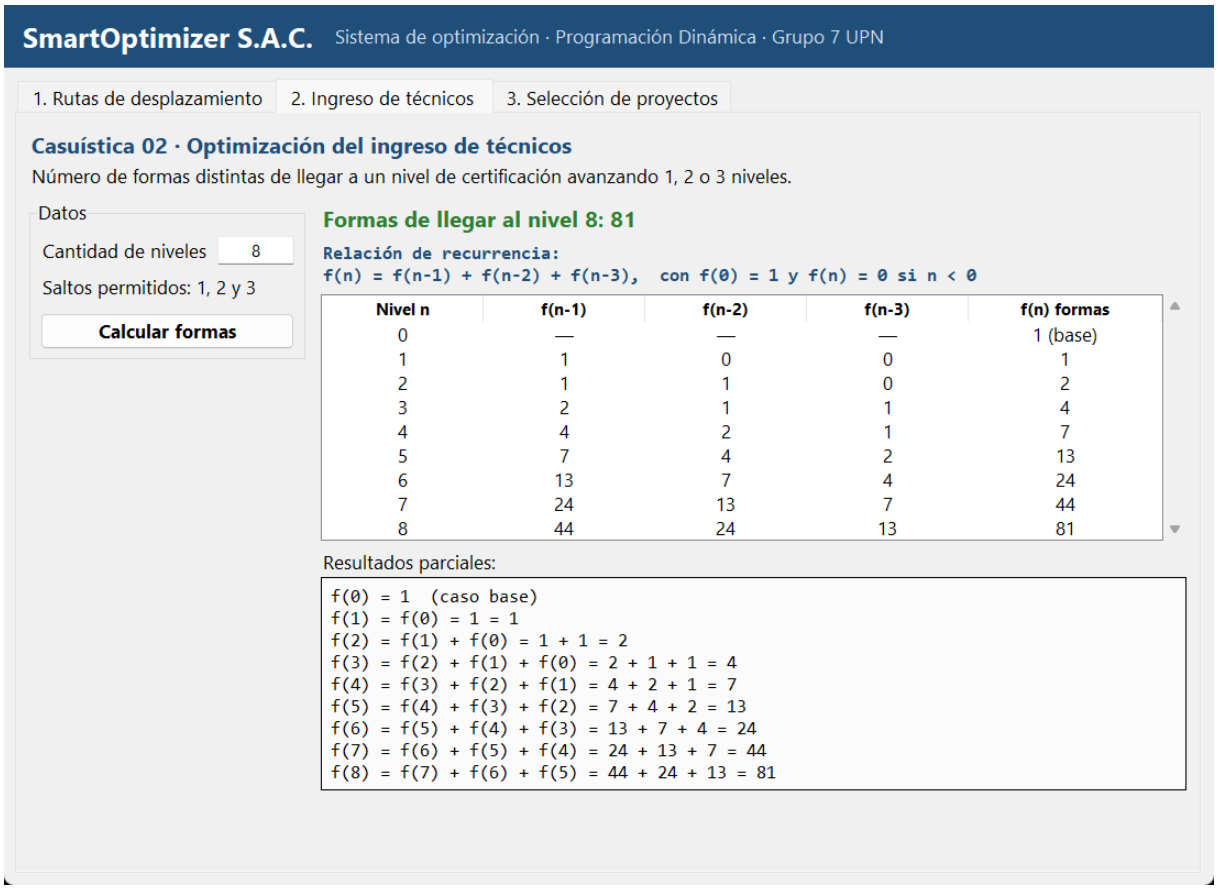
Nivel n	f(n−1)	f(n−2)	f(n−3)	f(n)
0	—	—	—	1 (base)
1	1	0	0	1
2	1	1	0	2
3	2	1	1	4
4	4	2	1	7
5	7	4	2	13
6	13	7	4	24
7	24	13	7	44
8	44	24	13	81

Nota. Elaboración propia.

Hay 81 formas distintas de llegar al nivel 8. Los resultados parciales muestran cómo cada valor se arma con los tres anteriores (por ejemplo, f(8) = 44 + 24 + 13 = 81). El número de formas casi se duplica con cada nivel, lo que indica que contar los caminos uno por uno sería cada vez más costoso.

Figura 2

Módulo 2 de la aplicación: recurrencia, tabla y resultados parciales



Nota. Elaboración propia.

3.4 Complejidad

La tabulación calcula cada f(i) una sola vez con 3 sumas: tiempo O(n) y espacio O(n), que podría reducirse a O(1) guardando solo los tres últimos valores. Una versión recursiva sin memoria repetiría los mismos subproblemas y tendría un costo cercano a $O(3^n)$.

4. Casuística 03: Selección óptima de proyectos tecnológicos

4.1 Planteamiento

SmartOptimizer tiene 8 proyectos con beneficios conocidos, pero no puede ejecutar dos consecutivos porque usan el mismo equipo especializado. Si dp[i] es el máximo beneficio usando solo los proyectos P1 a Pi, para el proyecto i hay dos opciones: no tomarlo y quedarse con dp[i−1], o tomarlo, lo que obliga a descartar Pi−1 y sumar su beneficio a dp[i−2]:

$$dp[i] = \max(\, dp[i-1],\ dp[i-2] + b[i]\,), \quad \text{con } dp[0] = 0 \text{ y } dp[1] = b[1]$$

4.2 Implementación

```
def seleccion_proyectos(beneficios):
    """Máximo beneficio sin elegir dos proyectos consecutivos.

    Recurrencia: dp[i] = max(dp[i-1],  dp[i-2] + b[i])
                 (no tomar el proyecto i  |  tomarlo y saltar el anterior)
    Caso base:   dp[0] = 0, dp[1] = b[1]
    Complejidad: O(n) en tiempo y O(n) en espacio.
    Devuelve (maximo, dp, seleccionados, descartados, filas) con índices 1..n.
    """
    n = len(beneficios)
    b = [0] + list(beneficios)            # b[1..n]
    dp = [0] * (n + 1)
    if n >= 1:
        dp[1] = b[1]
    filas = []                             # para mostrar la tabla DP paso a paso
    if n >= 1:
        filas.append((1, b[1], 0, b[1], dp[1], "Tomar"))
    for i in range(2, n + 1):
        sin_i = dp[i - 1]
        con_i = dp[i - 2] + b[i]
        dp[i] = max(sin_i, con_i)
        filas.append((i, b[i], sin_i, con_i, dp[i], "Tomar" if con_i > sin_i else "Saltar"))

    # Reconstrucción: si dp[i] viene de tomar i, se elige i y se salta a i-2
    seleccionados, i = [], n
    while i >= 1:
        if i == 1 or dp[i - 2] + b[i] > dp[i - 1]:
            if i == 1 and dp[1] == 0:
                break
            seleccionados.append(i)
            i -= 2
        else:
            i -= 1
    seleccionados.reverse()
    descartados = [i for i in range(1, n + 1) if i not in seleccionados]
    return dp[n], dp, seleccionados, descartados, filas
```

4.3 Resultados

Tabla 3

Tabla DP de la selección de proyectos (S/)

i	Proyecto	Beneficio	Sin Pi: dp[i−1]	Con Pi: dp[i−2] + b[i]	dp[i]	Decisión
1	P1	4,000	0	4,000	4,000	Tomar
2	P2	2,500	4,000	2,500	4,000	Saltar
3	P3	5,000	4,000	9,000	9,000	Tomar
4	P4	3,500	9,000	7,500	9,000	Saltar
5	P5	6,000	9,000	15,000	15,000	Tomar
6	P6	4,500	15,000	13,500	15,000	Saltar
7	P7	7,000	15,000	22,000	22,000	Tomar
8	P8	3,000	22,000	18,000	22,000	Saltar

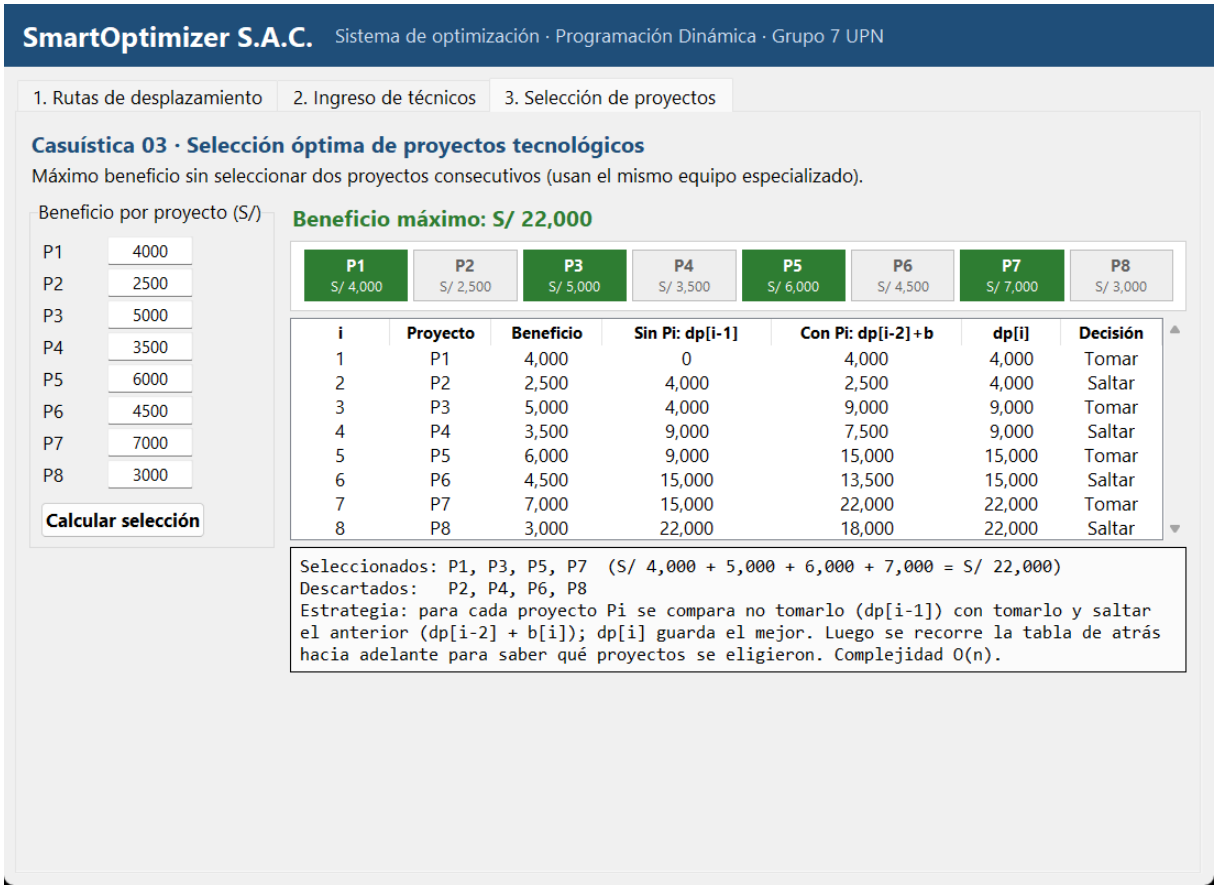
Nota. Elaboración propia.

El beneficio máximo es S/ 22,000, seleccionando P1, P3, P5, P7 (S/ 4,000 + 5,000 + 6,000 + 7,000). Se descartan P2, P4, P6, P8. Ningún par de proyectos elegidos es consecutivo, así que la restricción del equipo especializado se cumple.

Estrategia: la tabla se llena de P1 a P8 comparando en cada paso las dos opciones. Después se recorre de atrás hacia adelante: si dp[i] vino de tomar Pi, el proyecto se marca como elegido y se salta a i − 2; si no, se pasa a i − 1. Por ejemplo, en P8 conviene saltar: tomarlo da dp[6] + 3 000 = 18 000, menos que los 22 000 que ya se tenían con P7.

Figura 3

Módulo 3 de la aplicación: proyectos elegidos, tabla DP y estrategia



Nota. Elaboración propia.

4.4 Complejidad

Cada proyecto se evalúa una vez con una comparación: tiempo O(n) y espacio O(n). Probar todas las combinaciones válidas por fuerza bruta costaría $O(2^n)$, lo que para 8 proyectos son 256 subconjuntos, y se volvería impracticable con algunas decenas de proyectos.

5. Conclusiones

- La aplicación resuelve las tres casuísticas: la ruta de 0 a 10 cuesta como mínimo S/ 85; hay 81 formas de llegar al nivel 8; y el beneficio máximo sin proyectos consecutivos es S/ 22,000 (P1, P3, P5 y P7).

- Las tres soluciones comparten el mismo patrón: definir qué significa cada celda de la tabla, escribir la relación de recurrencia, fijar los casos base y llenar la tabla de menor a mayor.
- La Programación Dinámica reduce problemas de costo exponencial por fuerza bruta a costo lineal, O(n) u O(n·k), porque cada subproblema se resuelve una sola vez.
- Guardar la decisión tomada en cada paso permite reconstruir la solución (la ruta o los proyectos elegidos), no solo su valor.

Recomendaciones

- Para instancias más grandes, guardar solo las últimas filas de la tabla reduce el espacio a O(1) en los casos 2 y 3.
- Si los costos de la casuística 1 cambian con frecuencia, conviene leerlos desde un archivo de configuración en lugar de escribirlos en la interfaz.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms*. (4th ed.). MIT Press.

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson Education.

Python Software Foundation. (2024). *tkinter: Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html

Anexo: código fuente

El código completo y las capturas están en el repositorio del curso: https://github.com/MijaelCAN/analisis-algoritmos-upn-2026/tree/main/PC5-Programacion-dinamica